KnowFeat: Knowledge-Guided Feature Engineering with Auditable Provenance via LLM Agents

Chengsong You*, Wangyue Li*, Weiqiao Que*, Qizhou Chen*, Wei Deng†, Feng Zhu‡, Xiaofeng He*
*East China Normal University, Shanghai, China
{51275901122, 52285901017}@stu.ecnu.edu.cn, weiqiaoque@gmail.com, 582476065@qq.com, hexf@cs.ecnu.edu.cn
†Southwestern University of Finance and Economics, Chengdu, China
dengwei@swufe.edu.cn
‡Digital Currency Department, Postal Savings Bank of China, Beijing, China
Jason_bank@126.com

***Abstract*—Automated feature engineering (AutoFE) is critical for improving the performance of machine learning models on tabular data. Recent LLM-based approaches can generate semantically meaningful features, yet they lack hierarchical structured domain knowledge, rigorous multi-stage verification, and auditable provenance. We propose KnowFeat, a knowledge-guided feature engineering framework that integrates hierarchical domain knowledge into an LLM-based agent to generate high-quality, interpretable features with full traceability. KnowFeat introduces three key innovations: (1) a *five-level knowledge hierarchy* ranging from schema-only metadata to risk indicators, detection rules, expert opinions, and court document evidence, organized in a structured format to prevent hallucination; (2) a *three-stage verification pipeline* (L1: code execution sandbox, L2: statistical quality filtering, L3: model effectiveness evaluation) that ensures only genuinely useful features are accepted; and (3) *auditable provenance*—every generated feature is accompanied by a provenance card that traces its creation to specific knowledge assets, reasoning chains, and computation steps, enabling the auditability required in regulated domains such as finance and healthcare. We evaluate KnowFeat on seven public datasets and a real-world anti-money laundering (AML) dataset with 200K transactions. On public benchmarks, KnowFeat achieves the best AUC on 4 of 7 datasets and the best average rank (1.4) among seven methods. On the AML dataset, 18 knowledge-guided features improve Recall by +2.1pp and F1 by +1.5pp over a strong 93-feature baseline, outperforming methods that add up to 1,031 features. Ablation studies confirm that domain knowledge is essential for consistent feature generation.**

***Index Terms*—Feature Engineering, Large Language Models, Domain Knowledge, Anti-Money Laundering, Tabular Data**

I. Introduction

Feature engineering is widely recognized as one of the most impactful steps in building machine learning models for tabular data [1]. Carefully designed features can substantially improve model accuracy, reduce overfitting, and provide interpretable signals for downstream tasks. However, manual feature engineering is labor-intensive, requires deep domain expertise, and does not scale across diverse applications.

To address this challenge, automated feature engineering (AutoFE) methods have been developed. Traditional AutoFE approaches such as OpenFE [2] and AutoFeat [3] apply systematic transformations—mathematical operations, aggregations, and feature crosses—to existing columns, then select the most predictive ones. These methods are purely data-driven: they explore a predefined transformation space without any understanding of what the features *mean* or why certain transformations might be useful for a given domain. In practice, understanding the semantic meaning of features is crucial in domains such as finance, healthcare, and marketing, where stakeholders must know *what* a feature represents before deploying it in production.

The emergence of large language models (LLMs) has opened a new paradigm for feature engineering. Methods such as CAAFE [4], FeatLLM [5], and LLM-FE [6] use LLMs to generate feature transformation code based on dataset descriptions. These approaches leverage the LLM's broad world knowledge to propose semantically meaningful features—for example, generating a "debt-to-income ratio" for a credit dataset without being explicitly told to do so. However, current LLM-based methods share three fundamental limitations:

(1) Lack of hierarchical, structured domain knowledge. Most existing methods rely primarily on the LLM's parametric knowledge, which is general-purpose and may be outdated or incomplete for specialized domains. Some recent work incorporates limited forms of domain context: LLM-FE [6] provides dataset descriptions as evolutionary prompts, and RiskTagger [7] annotates Web3 behaviors using regulatory knowledge. However, none of these methods organizes domain knowledge into a *hierarchical, structured* format—from regulatory indicators and detection rules to expert opinions and case evidence—that enables systematic injection and prevents LLM hallucination. In high-stakes applications such as anti-money laundering (AML), such structured knowledge exists in external sources that LLMs cannot access through their training data alone.

(2) Insufficient verification. Existing verification mechanisms vary in rigor but remain incomplete. CAAFE evaluates candidates with a logistic regression proxy that poorly approximates the downstream model. LLM-FE uses downstream model accuracy as feedback but lacks statistical quality filtering. FeatLLM performs no model-based validation. None of these methods combines code execution checking, statistical

Corresponding author: hexf@cs.ecnu.edu.cn

quality filtering (e.g., information value, redundancy), and full model evaluation into a unified pipeline, leading to the acceptance of noisy, redundant, or harmful features.

(3) Limited auditability. In regulated industries such as finance, healthcare, and legal compliance, model features must be auditable. Decision-makers need to understand *why* a feature was created and *what evidence* supports its relevance. While some methods provide natural language explanations (e.g., RiskTagger generates annotation rationales), they lack fine-grained, structured provenance that traces each feature back to specific knowledge assets with detailed reasoning chains. Such auditable provenance is essential for deployment in environments that require explainability and regulatory accountability.

In this paper, we propose **KnowFeat** (**Know**ledge-guided **Feat**ure engineering), a framework that addresses all three limitations. KnowFeat has three core components:

- **Hierarchical Knowledge Injection.** We organize domain knowledge into five levels of increasing specificity: (i) schema-only metadata, (ii) risk indicators, (iii) detection rules, (iv) expert opinions, and (v) evidence extracted from court documents. This hierarchy is *ablatable*: we can systematically study the contribution of each knowledge level. The LLM agent receives the relevant knowledge as structured context and is instructed to ground every generated feature in specific knowledge assets.
- **Three-Stage Verification Pipeline.** Each candidate passes through L1 (code execution sandbox), L2 (statistical quality: IV, KS statistic, redundancy, missing rate, with noise-calibrated thresholds), and L3 (model effectiveness via XGBoost across 5 seeds), ensuring only genuinely useful features are accepted.
- **Auditable Provenance.** Every generated feature includes a structured provenance card recording the knowledge assets that inspired it, a reasoning chain linking domain knowledge to the design, computation steps, and verification results. This traceability enables compliance officers to audit *why* each feature exists—a requirement in regulated domains such as finance and healthcare.

We evaluate KnowFeat on seven public tabular datasets from OpenML and a real-world AML dataset containing 200,000 digital currency transactions with 93 baseline features. We compare against six baselines spanning traditional AutoFE (OpenFE, AutoFeat), LLM-based (CAAFE, FeatLLM), and evolutionary LLM (LLM-FE) approaches. Key findings include:

- On public datasets, KnowFeat achieves the highest AUC on 4 of 7 datasets and the best average rank (1.4) among all seven methods, while requiring significantly fewer features than alternative LLM-based methods (12–46 vs. 98–1,124 for FeatLLM).
- On the AML dataset, 18 knowledge-guided features improve Recall by +2.1pp and F1 by +1.5pp over a strong 93-feature baseline, substantially outperforming all baselines including LLM-FE (+1.4pp F1).
- Ablation experiments confirm that domain knowledge is essential: the schema-only variant generates zero accepted features on one dataset, and the full knowledge hierarchy outperforms schema-only on 5 of 6 remaining datasets.
- Every generated feature includes a provenance card linking it to specific knowledge assets, providing full auditability.

The remainder of this paper is organized as follows. Section II reviews related work. Section III presents the KnowFeat framework. Section IV describes our experimental setup and results. Section V illustrates KnowFeat's knowledge-grounded features through case studies. Section VI concludes with limitations and future directions.

II. Related Work

A. Traditional Automated Feature Engineering

Automated feature engineering aims to reduce the manual effort of creating predictive features from raw data. Early approaches focus on exhaustive transformation search: Nargesian et al. [8] propose learning-based feature engineering for classification, Deep Feature Synthesis [9] recursively applies aggregation and transformation primitives along relational paths, while ExploreKit [10] enumerates unary, binary, and higher-order operators over existing features and uses meta-learning to rank candidates. AutoFeat [3] constructs features through polynomial and interaction terms, then applies L1 regularization for selection. More recently, OpenFE [2] frames feature generation as an operator-operand search problem, using a two-stage expansion-and-selection pipeline with LightGBM-based feature importance to achieve state-of-the-art performance on tabular benchmarks.

These methods share a common limitation: they operate in a *syntactic* transformation space, applying predefined mathematical operators without any semantic understanding of the data domain. For example, computing `age × income` is treated identically to `age × zip_code`, even though only the former has financial meaning. This leads to a combinatorial explosion of candidates, most of which are semantically meaningless, and an inability to generate domain-specific features that require conceptual understanding—for instance, "transaction velocity" or "dormant account reactivation" in the context of transaction anomaly detection.

B. LLM-Based Feature Engineering

The emergence of large language models [11], [12] has enabled a paradigm shift from syntactic to semantic feature generation. CAAFE [4] iteratively prompts an LLM with dataset descriptions and evaluation feedback to generate Python code for new features, using logistic regression as a proxy evaluator. FeatLLM [5] takes a two-pass approach: first extracting natural language rules from the LLM, then converting these rules into binary indicator features. LLM-FE [6] frames feature engineering as an evolutionary optimization problem, using LLMs to iteratively generate and refine features through structured prompts with model performance feedback. In the financial domain, RiskTagger [7] uses an LLM agent

with regulatory knowledge to annotate Web3 crypto laundering behaviors, demonstrating that domain knowledge can enhance LLM-based data annotation.

These methods vary in their use of domain knowledge and verification. LLM-FE and RiskTagger incorporate limited domain context (dataset descriptions and regulatory guidelines, respectively), but none organizes knowledge into a hierarchical, structured format for systematic injection. Verification also varies: CAAFE uses a weak proxy model, LLM-FE uses downstream model accuracy, and FeatLLM performs no model-based validation. None combines code execution, statistical quality filtering, and model evaluation into a unified pipeline. Furthermore, while RiskTagger provides natural language rationales, none offers structured provenance that traces each feature to specific knowledge assets.

Tool-augmented LLM agents [13] have been applied to code generation [14], data analysis [15], and scientific discovery [16]. In data science, AIDE [17] and DS-Agent [18] automate end-to-end pipelines but do not address knowledge-grounded feature generation. KnowFeat builds on the tool-use paradigm by equipping an LLM agent with specialized tools for data exploration, feature generation with provenance, and multi-stage verification.

Table I summarizes the key differences. KnowFeat is the first method to combine hierarchical structured domain knowledge, multi-stage verification (L1–L3), and full feature provenance tracking within an LLM-based feature engineering framework.

TABLE I
COMPARISON OF AUTOMATED FEATURE ENGINEERING METHODS.

Property	**OpenFE**	**Auto. Feat.**	**CAAFE**	**LLM-FE**	**RiskTag.**	**Ours**
Semantic underst.			✓	✓	✓	✓	✓
Hier. knowl.					Partial	Partial	✓
Exec. verific.	–	–					✓
Statistical filt.	✓	✓					✓
Model validation			Weak		✓		✓
Provenance track.						NL only	✓
Adaptive search					✓		✓

C. Domain Knowledge Integration in Machine Learning

The integration of domain knowledge into machine learning has a long history. Knowledge-infused learning [19] incorporates external knowledge graphs into neural architectures. In the feature engineering context, domain experts traditionally encode their knowledge as hand-crafted features—a process that is effective but does not scale. Transfer learning approaches [20] implicitly transfer knowledge through pretrained representations, but these are primarily designed for unstructured data (text, images) rather than tabular features.

In the financial domain specifically, AML feature engineering relies on expert-defined indicators such as transaction velocity, structuring patterns, and network centrality measures [21]. Recent work has also explored graph-based approaches for financial forensics [22]. Regulatory bodies publish typologies and red-flag indicators (e.g., FATF guidelines [23]), and financial institutions maintain internal detection rules derived from compliance experience and legal proceedings. This rich, structured knowledge has not been systematically leveraged by automated feature engineering methods.

III. METHODOLOGY

This section presents the KnowFeat framework in detail. We first provide an overview of the pipeline (Section III-A), then describe each component: hierarchical knowledge injection (Section III-B), the LLM agent with specialized tools (Section III-C), the three-stage verification pipeline (Section III-D), the intelligent search strategy (Section III-E), and auditable provenance output (Section III-F).

A. Framework Overview

KnowFeat is a multi-round, knowledge-guided feature engineering framework built on an LLM agent. Given a tabular dataset $\mathcal{D} = \{(\mathbf{x}_i, y_i)\}_{i=1}^{N}$ with baseline features $\mathbf{x}_i \in \mathbb{R}^d$ and target label y_i, along with a domain knowledge base $\mathcal{K}$, KnowFeat generates a set of new features $\mathcal{F}^* = \{f_1, f_2, \ldots, f_k\}$ such that the augmented dataset $\mathcal{D}' = \{([\mathbf{x}_i;\ f_1(\mathbf{x}_i);\ \cdots\ ;\ f_k(\mathbf{x}_i)],\ y_i)\}$ achieves improved predictive performance on a target metric (e.g., AUC [24] or Recall).

The overall pipeline operates as follows (see Figure 1 for an illustration):

1) **Initialization.** Load the dataset, compute baseline model performance, and calibrate statistical thresholds via noise injection (Section III-D).
2) **Knowledge Assembly.** Organize domain knowledge $\mathcal{K}$ into a five-level hierarchy and inject the selected level into the agent's context (Section III-B).
3) **Multi-Round Generation.** For each round $r = 1, \ldots, R_{\max}$:
   a) Allocate per-dimension budgets based on SHAP feedback from the previous round.
   b) For each semantic dimension (Section III-E), the LLM agent explores the data, proposes candidate features grounded in knowledge assets, and submits them for verification.
   c) Each candidate passes through the L1→L2→L3 pipeline. Accepted features are added to the baseline; rejected features enter the failure memory.
   d) Forward selection discovers feature combinations that are individually weak but collectively effective.
4) **Convergence.** The process terminates when no new features are accepted for C consecutive rounds (default $C = 2$) or the maximum round count $R_{\max}$ is reached.
5) **Output.** The accepted feature set $\mathcal{F}^*$, along with executable code and auditable provenance cards for each feature (Section III-F).

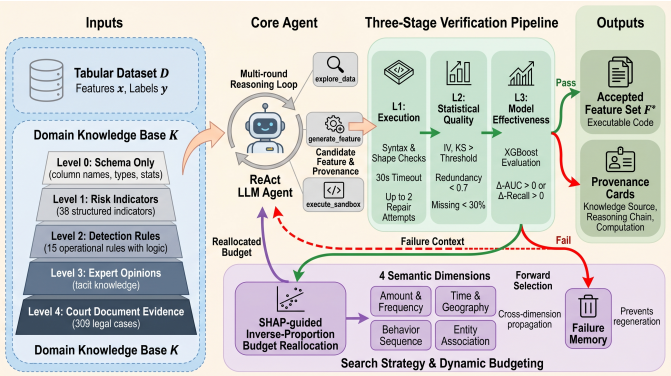


Fig. 1. Overview of the KnowFeat framework. The LLM agent generates features guided by hierarchical domain knowledge. Each candidate is verified through a three-stage pipeline (L1–L3). SHAP feedback dynamically reallocates the exploration budget across semantic dimensions.

B. Hierarchical Knowledge Injection

A key innovation of KnowFeat is the systematic integration of external domain knowledge into the feature generation process. We organize knowledge into five hierarchical levels of increasing specificity:

Level 0: Schema Only. The agent receives only the dataset schema—column names, data types, and basic statistics (mean, cardinality, missing rate). No domain knowledge is provided. This serves as the ablation baseline, testing what the LLM can generate from metadata alone.

Level 1: Risk Indicators. Structured risk indicators derived from regulatory guidelines and industry standards. Each indicator specifies a risk dimension, risk class, relevant data field, and directionality. For the AML domain, we compile 38 indicators (IND_01 through IND_38) covering transaction amount thresholds, frequency patterns, temporal anomalies, and network centrality measures. For example, IND_05 flags accounts whose single-day transaction count exceeds a regulatory threshold, while IND_15 identifies abnormal transaction modes such as fan-in and many-to-many patterns.

Level 2: Detection Rules. Operational detection rules that encode specific logic patterns. Each rule includes a rule ID, natural language description, logical expression, confidence level, pattern category, and a Python implementation hint. We compile 15 rules (RULE_001 through RULE_015) from compliance manuals and internal documentation. For example, RULE_006 encodes the "scatter-in, gather-out" layering pattern where funds are collected from many sources then concentrated for withdrawal.

Level 3: Expert Opinions. Unstructured insights from domain expert interviews. These capture tacit knowledge—heuristics, edge cases, and domain intuitions—that are difficult to formalize as rules but valuable for feature design. We collect 12 expert opinions (EXP_01 through EXP_12) encoded as natural language paragraphs. For example, EXP_05 notes that "border region transactions combined with nighttime activity are a hallmark of cross-border laundering."

Level 4: Court Document Evidence. Risk factors extracted from real legal proceedings using a key-clue extraction pipeline. For the AML domain, we process 309 court judgment documents to extract evidence-grounded risk factors. This represents the most concrete and specific level of domain knowledge, grounding feature generation in real-world case evidence. For example, court documents reveal that intermediate wallets in pyramid-shaped fund networks exhibit strong fan-in/fan-out asymmetry—a pattern that informs Level 2 rules but is independently validated by legal proceedings.

The five levels can be freely combined: by setting a knowledge filter parameter, any subset of levels can be included, enabling systematic study of each level's contribution through ablation. In the "full" configuration (default), all levels are included. The knowledge base is formatted as structured text and injected into the agent's system prompt, with each knowledge item assigned a unique identifier for provenance tracking.

C. LLM Agent with Specialized Tools

KnowFeat employs an LLM agent (GPT-4o in our implementation) that interleaves reasoning with tool calls following the ReAct paradigm [13]. The agent is equipped with three specialized tools:

Tool 1: Data Exploration. Executes Python code on the dataset in a sandboxed environment to inspect distributions, compute correlations, and identify patterns before proposing features. The tool returns printed output or return values to the agent.

Tool 2: Feature Registration. Registers a candidate feature with mandatory provenance fields:

- `feature_name`: A descriptive identifier.
- `code`: Executable Python code that computes the feature from raw data.
- `explanation`: A natural language description of what the feature captures.
- `reasoning`: A structured chain linking knowledge assets to the feature design, formatted as "Step N: Because [knowledge ID] indicates [insight], we compute [transformation]..."
- `knowledge_ids`: A list of specific knowledge asset identifiers (e.g., RULE_001, IND_05, EXP_03) that inspired the feature.
- `computation_steps`: Step-by-step description from raw fields to the final feature value.
- `data_fields_used`: The raw columns consumed by the feature.

Tool 3: Verification Trigger. Triggers the L1→L2→L3 verification pipeline on a registered feature and returns the result (accepted/rejected with detailed reasons). The agent must call Feature Registration followed immediately by Verification Trigger for each candidate—batch submission is prohibited to ensure tight feedback loops.

The agent's prompt instructs it to: (1) reason about which knowledge assets are relevant to the current dimension and budget, (2) explore the data to validate assumptions, (3)

propose a feature grounded in specific knowledge items, and (4) submit it for verification and react to the feedback.

D. Three-Stage Verification Pipeline

Each candidate feature passes through three progressively stricter verification stages:

1) L1: Code Execution Sandbox: The generated Python code is executed in an isolated sandbox with a 30-second timeout. L1 checks for:

- Syntax and runtime errors.
- Output shape consistency (one value per sample).
- Presence of required identifier columns.
- Data type validity (numeric output).

If L1 fails, the agent receives the error message and is allowed up to 2 automatic repair attempts before the feature is permanently rejected.

2) L2: Statistical Quality Filtering: The feature's statistical properties are evaluated against automatically calibrated thresholds. To establish these thresholds for discriminative power, we generate $N_{\text{noise}} = 100$ random features (drawn from standard distributions), compute their quality metrics, and set the acceptance threshold at the P_{noise}-th percentile (default $P_{\text{noise}} = 95$). This ensures that accepted features are statistically better than random noise. L2 checks four criteria:

- **Information Value (IV)** [21]: Must exceed the noise-calibrated threshold, ensuring the feature has meaningful discriminative power between positive and negative classes.
- **Kolmogorov–Smirnov (KS) Statistic** [8]: Must exceed the noise-calibrated threshold, measuring the maximum distributional separation between classes.
- **Redundancy**: Maximum absolute Pearson correlation with all existing accepted features must be below 0.7, preventing near-duplicate features from inflating the feature set.
- **Missing Rate**: The proportion of NaN values must be at most 50%, ensuring sufficient coverage for model training.

The noise-calibrated thresholds apply to IV and KS; the redundancy and missing rate thresholds are fixed.

3) L3: Model Effectiveness: The feature is evaluated for its actual contribution to predictive performance. We train an XGBoost [25] model on the baseline features augmented with the candidate feature, using stratified 80/20 train-test splits across 5 random seeds $\{42, 123, 456, 789, 2024\}$. The acceptance criterion depends on the task:

- **Classification (public datasets)**: $\Delta\text{AUC} > 0$, i.e., the augmented model must achieve strictly higher AUC than the baseline.
- **AML detection**: $\Delta\text{Recall} > 0$ *and* $\text{Precision} \geq 90\%$, ensuring that the feature improves detection without sacrificing precision.

The three-stage design is intentionally progressive: L1 is cheap (milliseconds), L2 is moderate (seconds), and L3 is expensive (minutes per feature). By filtering early, we avoid wasting computational resources on clearly defective candidates.

E. Intelligent Search Strategy

To systematically explore the feature space, KnowFeat employs a multi-component search strategy.

1) Dimension-Based Budgeting: We decompose the feature space into D semantic dimensions relevant to the target domain (default $D = 4$ for the AML domain):

1) **Amount & Frequency**: Features related to monetary values and transaction counts.
2) **Time & Geography**: Features capturing temporal patterns and spatial distributions.
3) **Behavior Sequence**: Features encoding sequential patterns in user actions.
4) **Entity Association**: Features derived from relationships between entities (accounts, counterparties).

Each dimension receives an initial budget of B_0 candidates per round (default $B_0 = 3$). After each round, SHAP [26] (SHapley Additive exPlanations) analysis is performed on the current model to estimate each dimension's contribution to predictions.

2) SHAP-Guided Budget Reallocation: Let s_j denote the mean absolute SHAP value for dimension j's accepted features. We reallocate the budget using an *inverse-proportion* strategy:

$$B_j^{(r+1)} = B_{\min} + (B_{\text{total}} - D \cdot B_{\min}) \cdot \frac{1/s_j}{\sum_{j'=1}^{D} 1/s_{j'}} \quad (1)$$

where $B_{\min}$ is a minimum per-dimension budget (default 1) and B_{total} is the total budget for the round. This ensures that underrepresented dimensions—those contributing least to the current model—receive more exploration budget in subsequent rounds, promoting balanced feature coverage.

3) Failure Memory: A failure memory $\mathcal{M}_{\text{fail}}$ records all rejected features along with their rejection reasons. Before each generation step, the K most recent failures for the current dimension (default $K = 10$) are injected into the agent's prompt, preventing the regeneration of previously rejected ideas and guiding the agent toward novel directions.

4) Forward Selection: Individual features may fail L3 yet contribute positively when combined with others. We apply greedy forward selection on L2-passed but L3-failed candidates, sorted by L2 quality scores. Each candidate is added to the accepted set if the augmented set improves the target metric, capturing synergistic feature interactions.

5) Cross-Dimension Propagation: After each round, the top-K_{cross} features (default $K_{\text{cross}} = 5$) by SHAP importance are shared across all dimensions as "inspiration prompts," encouraging the agent to adapt successful patterns from one dimension to others.

F. Auditable Provenance Output

A distinguishing contribution of KnowFeat is that every generated feature is accompanied by a structured *provenance*

TABLE II
DATASET STATISTICS.

Dataset	Samples	Feat.	Pos%	Domain
credit_g	1,000	20	70.0	Finance
diabetes	768	8	34.9	Healthcare
blood_transfusion	748	4	23.8	Healthcare
heart	918	11	55.3	Healthcare
bank_marketing	45,211	16	11.7	Marketing
breast_w	683	9	35.0	Healthcare
adult	48,842	14	23.9	Census
SimECNY (AML)	200,000	93	2.24	Finance

card that enables full auditability. Each provenance card records:

- **Knowledge assets**: The specific identifiers (e.g., RULE_006, IND_15, EXP_05) that inspired the feature design.
- **Reasoning chain**: A step-by-step explanation of how domain knowledge led to the feature, formatted as "Step N: Because [knowledge ID] indicates [insight], we compute [transformation]…"
- **Computation steps**: The transformation pipeline from raw data fields to the final feature value.
- **Verification results**: L1/L2/L3 outcomes including IV, KS, redundancy, and ΔAUC/ΔRecall.

This traceability is essential for regulated domains: compliance officers can audit *why* each feature exists, *what evidence* supports it, and *how* it was verified. Section V illustrates provenance cards through concrete examples.

IV. EXPERIMENTS

We evaluate KnowFeat on both public tabular benchmarks and a real-world anti-money laundering (AML) dataset. Our experiments aim to answer four research questions:

- **RQ1**: Does KnowFeat improve predictive performance on public datasets?
- **RQ2**: How does KnowFeat perform on a real-world AML task with severe class imbalance?
- **RQ3**: What is the contribution of domain knowledge?
- **RQ4**: How does KnowFeat balance feature quantity vs. quality?

A. Experimental Setup

1) Datasets: We use eight datasets summarized in Table II: seven public benchmarks from OpenML [24] and one real-world AML dataset.

The seven public datasets are standard tabular classification benchmarks obtained via the OpenML Python API [24]. SimECNY is a real-world digital currency transaction dataset containing 200,000 transactions with 93 baseline features and 4,484 positive (suspicious) samples, representing a realistic AML scenario with severe class imbalance (2.24% positive rate).

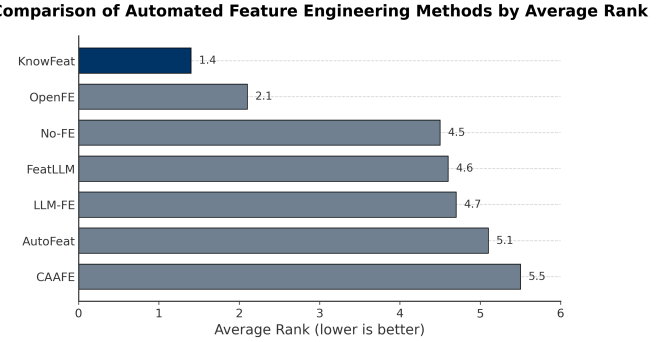


Fig. 2. Average rank across 7 public datasets (lower is better). KnowFeat achieves the best average rank (1.4).

2) Baselines: We compare against six baselines:

- **No-FE**: XGBoost [25] on original features.
- **OpenFE** [2]: Transformation-based AutoFE with LightGBM [27]-based selection.
- **AutoFeat** [3]: Polynomial feature generation with L1 selection.
- **CAAFE** [4]: LLM-based iterative feature generation with logistic regression feedback.
- **FeatLLM** [5]: LLM-based rule extraction and binary feature generation.
- **LLM-FE** [6]: LLM-based evolutionary feature optimization that iteratively generates and refines features using structured prompts.

3) Evaluation Protocol: All methods are evaluated using XGBoost [25] (n_estimators=200, max_depth=6, learning_rate=0.1) with stratified 80/20 train-test splits across 5 random seeds. We use AUC as the primary metric for all datasets, as it is threshold-independent and more robust than accuracy for datasets with varying class distributions. For SimECNY, we additionally report Recall, Precision, and F1 with scale_pos_weight=auto ($\approx$43.6) to handle class imbalance. All baselines are reproduced using our unified evaluation protocol to ensure fair comparison.

B. RQ1: Public Dataset Performance

Table III presents the AUC comparison on seven public datasets. KnowFeat achieves the best performance on 4 out of 7 datasets (credit_g, blood_transfusion, breast_w, and adult) and ranks second on 2 datasets (diabetes and heart).

Figure 2 visualizes the average ranks. KnowFeat achieves the best average rank (1.4), substantially outperforming the second-best OpenFE (2.1). The largest improvements occur on datasets where domain knowledge provides strong semantic guidance: credit_g (+0.018 over No-FE) and blood_transfusion (+0.018 over No-FE).

OpenFE achieves the best results on 3 datasets (diabetes, heart, bank_marketing) through exhaustive operator-operand search. However, OpenFE generates 34–50 features per dataset, while KnowFeat achieves competitive or superior performance with 12–46 features (see Section IV-E).

TABLE III
AUC COMPARISON ON PUBLIC DATASETS (MEAN±STD OVER 5 SEEDS). **BOLD**: BEST; UNDERLINE: SECOND BEST. T/O = TIMEOUT.

Dataset	No-FE	OpenFE	AutoFeat	CAAFE	FeatLLM	LLM-FE	KnowFeat
credit_g	.7670±.021	.7833±.031	.7540±.017	.7596±.023	.7604±.022	.7615±.026	**.7854±.014**
diabetes	.8027±.033	**.8252±.034**	.8089±.027	.8032±.030	.8096±.032	.8148±.033	.8190±.032
blood_transfusion	.6824±.043	.6654±.040	.6824±.043	.6779±.043	.6805±.044	.6649±.045	**.7007±.036**
heart	.9320±.007	**.9417±.011**	.9340±.003	.9302±.006	.9297±.004	.9330±.005	.9379±.006
bank_marketing	.9330±.002	**.9374±.002**	T/O	.9334±.002	.9337±.002	.9317±.002	.9344±.002
breast_w	.9926±.005	.9944±.005	.9936±.004	.9924±.006	.9924±.005	.9939±.001	**.9957±.003**
adult	.9288±.004	.9300±.003	.9254±.004	.9286±.003	.9289±.003	.9275±.003	**.9301±.003**
Avg. Rank	4.5	2.1	5.1	5.5	4.6	4.7	**1.4**

TABLE IV
SIMECNY AML RESULTS (SPW=AUTO, 5 SEEDS).

Method	n_f	Recall	Prec.	F1	AUC
No-FE	93	.9663±.010	.9764±.004	.9713±.005	.9996
OpenFE	123	.9645±.008	.9752±.004	.9698±.003	.9996
CAAFE	94	.9681±.008	.9725±.006	.9703±.005	.9996
FeatLLM	1124	.9625±.010	.9728±.005	.9676±.006	.9995
LLM-FE	94	.9699±.007	.9758±.004	.9728±.005	.9996
Ours	**111**	**.9871**±.006	**.9855**±.001	**.9863**±.003	**.9999**

TABLE V
ABLATION: SCHEMA_ONLY VS. FULL KNOWFEAT (AUC).

Dataset	schema_only	full	Δ
credit_g	.7750±.012	**.7854**±.014	+.010
blood_trans.	.6891±.048	**.7007**±.036	+.012
heart	.9304±.005	**.9379**±.006	+.008
bank_mkt.	.9314±.002	**.9344**±.002	+.003
adult	.9292±.003	**.9301**±.003	+.001
breast_w	**.9964**±.002	.9957±.003	−.001
diabetes	— (0 feat.)	.8190±.032	—

Among LLM-based methods, LLM-FE generally outperforms CAAFE and FeatLLM but remains below both OpenFE and KnowFeat. Notably, LLM-FE underperforms the No-FE baseline on bank_marketing (.9317 vs. .9330) and blood_transfusion (.6649 vs. .6824), suggesting that evolutionary prompt optimization alone, without external domain knowledge and multi-stage verification, is insufficient to consistently improve over raw features.

C. RQ2: AML Dataset Performance

Table IV presents results on the SimECNY AML dataset with scale_pos_weight=auto. KnowFeat adds 18 knowledge-guided features to the 93-feature baseline, achieving the best results across all four metrics.

KnowFeat's 18 domain-knowledge features yield substantial improvements: Recall improves by +2.1pp over No-FE (0.9871 vs. 0.9663), F1 by +1.5pp (0.9863 vs. 0.9713), and AUC reaches 0.9999. LLM-FE achieves the second-best F1 (0.9728) with only 1 additional feature, but KnowFeat's margin over LLM-FE remains large (+1.4pp F1, +1.7pp Recall). FeatLLM adds 1,031 features yet achieves the *worst* performance across all metrics.

To verify that KnowFeat's gains stem from intrinsic feature quality rather than interaction with class balancing, we re-evaluate all methods with scale_pos_weight=1 (no class balancing). KnowFeat remains the clear winner (Recall 0.9683, F1 0.9812), and is the *only* method that outperforms the No-FE baseline. KnowFeat suffers the smallest Recall drop when switching from spw=auto to spw=1 ($\Delta = -1.9$pp), while FeatLLM drops the most ($\Delta = -3.3$pp). This confirms that KnowFeat's 18 features encode genuine discriminative signal that does not depend on class reweighting.

D. RQ3: Ablation Study

To quantify the contribution of domain knowledge, we compare the *schema_only* variant (Level 0: only dataset metadata) against the *full* variant (all knowledge levels). Results on public datasets are shown in Table V.

Knowledge drives feature generation. On the diabetes dataset, the schema-only variant generates *zero* features that pass L3 verification. With full knowledge, KnowFeat generates 24 accepted features and improves AUC by +0.016 over No-FE.

Knowledge improves feature quality. On five of the six remaining datasets, the full variant outperforms schema_only, with gains ranging from +0.001 (adult) to +0.012 (blood_transfusion). On blood_transfusion, both variants generate 12 features, indicating that the improvement comes from feature *quality*, not quantity.

SimECNY ablation. On the AML dataset, domain knowledge is critical: schema_only generates only 3 features (+0.3pp Recall over No-FE), while the full knowledge hierarchy generates 18 features (+2.1pp Recall). The gap between schema_only and full (+1.8pp Recall, +1.4pp F1) confirms that the AML-specific knowledge assets (detection rules, behavioral patterns, expert opinions) are the primary driver of KnowFeat's advantage.

E. RQ4: Feature Quantity vs. Quality

Table VI compares the number of features generated by each method.

TABLE VI
NUMBER OF FEATURES (INCL. ORIGINAL).

Dataset	No-FE	Open. FE	Auto. Feat	CAA. FE	Feat. LLM	LLM- FE	Ours
credit_g	20	50	23	24	280	21	46
diabetes	8	38	13	9	152	15	24
blood_trans.	4	34	4	7	120	9	12
heart	11	41	21	14	98	19	31
bank_mkt.	16	46	T/O	21	132	20	36
breast_w	9	39	20	10	107	15	16
adult	14	44	165	19	170	21	34
SimECNY	93	123	—	94	1124	94	111

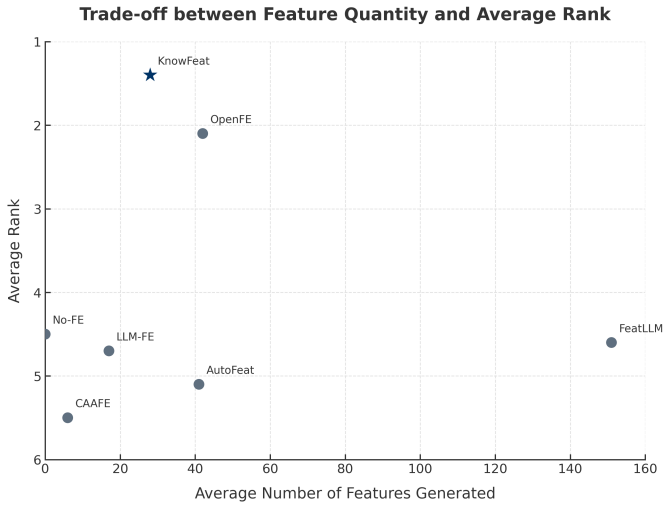


Fig. 3. Trade-off between feature quantity and average rank. KnowFeat achieves the best rank with a moderate number of verified features.

FeatLLM generates by far the most features (98–1,124) yet achieves the worst or near-worst performance on most datasets. AutoFeat generates 165 features on adult yet ranks last (AUC 0.9254). LLM-FE generates a conservative number of features (9–21) but cannot consistently outperform the No-FE baseline.

Figure 3 visualizes this trade-off. KnowFeat strikes an effective balance: it generates a moderate number of features (12–46 on public datasets, +18 on SimECNY) that have all passed the L1→L2→L3 pipeline. On SimECNY, the contrast is particularly stark: KnowFeat's 18 verified features improve F1 by +1.5pp, while FeatLLM's 1,031 unverified features *decrease* F1 by −0.4pp.

F. Provenance and Interpretability

Every KnowFeat-generated feature includes a provenance card recording: (1) the knowledge assets that inspired it; (2) a reasoning chain explaining *why* the feature is relevant; and (3) the computation steps from raw fields to the final value. This traceability is essential for deployment in regulated industries. We illustrate this capability through detailed case studies in Section V.

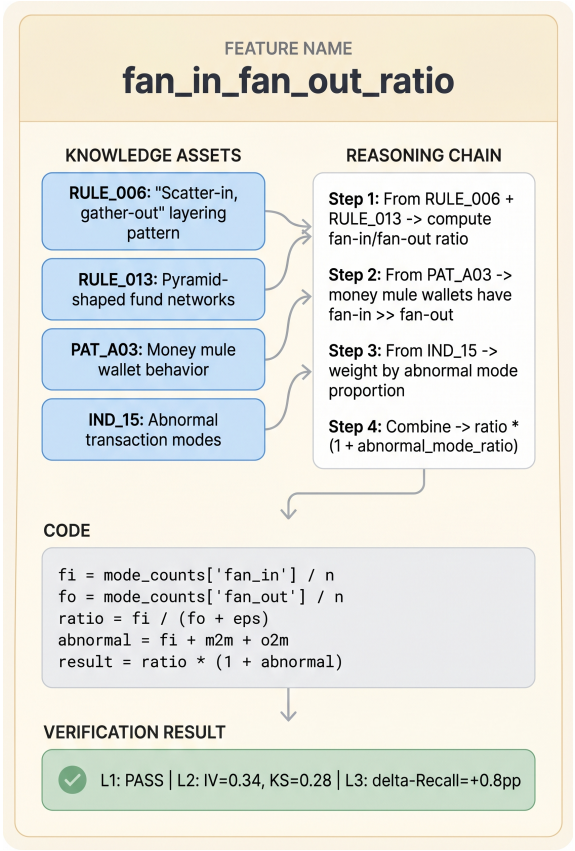


Fig. 4. Provenance card for the `fan_in_fan_out_ratio` feature, showing knowledge assets, reasoning chain, generated code, and verification results.

V. CASE STUDY: KNOWLEDGE-GROUNDED FEATURES

To illustrate how KnowFeat leverages domain knowledge to generate interpretable, high-quality features, we present three representative case studies drawn from the SimECNY AML dataset. Each example highlights a different aspect of knowledge-guided feature engineering: multi-source knowledge fusion, domain-specific behavioral patterns, and cross-dimensional interaction. Figure 4 shows an example provenance card.

A. Case 1: Fan-In/Fan-Out Ratio (Network Topology)

Feature: `fan_in_fan_out_ratio`, which measures the asymmetry between incoming and outgoing transaction patterns, weighted by the proportion of abnormal transaction modes.

Knowledge assets used:

- *RULE_006* (Detection Rule): "Scatter-in, gather-out," a classic layering pattern where funds are collected from many sources then concentrated for withdrawal.
- *RULE_013* (Detection Rule): Pyramid-shaped fund networks where intermediate wallets exhibit strong fan-in/fan-out asymmetry.
- *PAT_A03* (Behavioral Pattern): Online money mule wallets receive from many senders (high fan-in) then forward to a few consolidation accounts.

- *IND_15* (Risk Indicator): Abnormal transaction modes (fan_in, many_to_many, one_to_many) are individually suspicious.

Reasoning chain:

1) From *RULE_006* and *RULE_013*: compute the ratio of fan-in to fan-out mode proportions per wallet.
2) From *PAT_A03*: money mule wallets have fan-in $\gg$ fan-out; this asymmetry ratio is the primary signal.
3) From *IND_15*: weight by total abnormal mode proportion, since wallets with more abnormal modes are higher risk.
4) Combine: ratio $\times$ (1 + abnormal_mode_ratio).

Generated code (simplified):

```
fi = mode_counts['fan_in'] / n
fo = mode_counts['fan_out'] / n
ratio = fi / (fo + eps)
abnormal = fi + m2m_ratio + o2m_ratio
result = ratio * (1 + abnormal)
```

Why this matters. This feature fuses knowledge from four assets spanning three types (rules, patterns, indicators) to capture network topology. No individual column contains "fan-in/fan-out asymmetry"—it requires understanding that the *combination* of directional imbalance and abnormal mode prevalence characterizes money mule behavior. A syntactic method might discover pairwise mode interactions but would not produce a topology-aware asymmetry score grounded in the "scatter-in, gather-out" layering pattern.

B. Case 2: Probe-Then-Transfer Ratio

Feature: `small_test_round_amount_ratio`, which detects the "probe-then-bulk-transfer" pattern by combining small-amount test ratio with round-number transfer ratio.

Knowledge assets used:

- *RULE_003* (Detection Rule): Rapid fund movement—small probe transactions ($\leq$5 CNY) precede large-value transfers.
- *PAT_A02/A03* (Behavioral Pattern): Money mule operations begin with "small test, then bulk operation."
- *IND_01/02* (Risk Indicator): Round-number amounts (multiples of 100) and repetitive amount patterns indicate structured transactions.

Reasoning chain:

1) From *RULE_003* and *PAT_A02/A03*: compute the proportion of transactions with amount $\leq$5 CNY (probe ratio).
2) From *IND_01/02*: compute the proportion of transactions with amounts divisible by 100 (round-number ratio).
3) From domain expertise (*EXP_01*): "probe + round-number bulk transfer" co-occurrence is far more discriminative than either signal alone.
4) Combine: small_ratio $\times$ (1 + round_100_ratio).

Generated code (simplified):

```
amt = abs(amount)
small_ratio = (amt <= 5).sum() / n
round_ratio = (amt % 100 == 0).sum() / n
result = small_ratio * (1 + round_ratio)
```

Why this matters. The "probe-then-transfer" pattern is well-documented in AML regulations but impossible to discover through syntactic transformation search. It requires understanding that sub-5-CNY transactions serve a fundamentally different purpose (channel testing) than regular small purchases. The specific threshold (5 CNY) and the interaction with round-number amounts come directly from *RULE_003* and compliance expertise—transformations that no data-driven method would enumerate.

C. Case 3: Cross-Dimensional Risk Interaction

Feature: `high_risk_region_off_hour_ratio`, a multiplicative interaction between geographic risk and temporal anomaly.

Knowledge assets used:

- *RULE_005/014* (Detection Rules): Cross-regional fund flows involving border areas (region codes 540000/650000) are high-risk for underground banking.
- *IND_08/09* (Risk Indicators): Transactions concentrated in off-hours (18:00–8:00) indicate abnormal activity.
- *EXP_05* (Expert Opinion): "Border region + nighttime trading is the hallmark of cross-border money laundering."

Reasoning chain:

1) From *RULE_005/014*: flag wallets registered in high-risk border regions.
2) From *IND_08/09*: compute off-hour transaction ratio.
3) From *EXP_05*: neither signal alone is sufficient—border wallets are not inherently suspicious, nor are off-hour transactions. The *combination* is the strong signal.
4) Combine via multiplication: is_high_risk $\times$ off_hour_ratio. Non-border wallets receive 0 regardless of timing.

Generated code (simplified):

```
is_hr = region.isin([540000, 650000])
off = (hour >= 18) | (hour < 8)
off_ratio = off.sum() / n
result = is_hr * off_ratio
```

Why this matters. This feature demonstrates *cross-dimensional interaction*: it combines geographic context (from account metadata) with temporal patterns (from transaction logs), guided by expert knowledge that explicitly identifies the combination as a "hallmark" signal. Syntactic methods cannot cross data sources or reason about why border + nighttime is meaningful. The multiplicative form ensures that neither dimension alone triggers a high score—only the conjunction, exactly as the expert knowledge prescribes.

D. Summary of Case Study Findings

Table VII summarizes the three case studies.

These examples demonstrate three capabilities that distinguish KnowFeat: (1) *multi-source fusion*—combining rules, behavioral patterns, and risk indicators into topology-aware

TABLE VII
SUMMARY OF CASE STUDY FEATURES.

Feature	#KA	Key Insight
`fan_in_fan_out`	4	Network topology fusion
`small_test_round`	5	Behavioral sequence pattern
`region_off_hour`	4	Cross-dim. interaction

KA = Knowledge Assets in provenance chain.

features; (2) *domain-specific patterns*—encoding regulatory knowledge (probe thresholds, structuring indicators) that syntactic search cannot discover; and (3) *cross-dimensional interaction*—combining signals from different data sources guided by expert knowledge about their conjunction. Every feature includes a complete provenance chain, enabling compliance officers to audit the feature's rationale.

VI. Conclusion

We presented KnowFeat, a knowledge-guided feature engineering framework that combines hierarchical domain knowledge injection, multi-stage verification (L1–L3), and auditable provenance within an LLM agent. On seven public datasets, KnowFeat achieves the best AUC on 4 of 7 and the best average rank (1.4) among seven methods, while requiring significantly fewer features than alternative LLM-based methods. On a real-world AML dataset with 200K transactions, 18 knowledge-guided features improve Recall by +2.1pp and F1 by +1.5pp over a strong 93-feature baseline—gains that are robust across different class balancing settings. The ablation study confirms that domain knowledge is essential, and every generated feature is traceable to specific knowledge assets.

Limitations and Future Work. Our current knowledge hierarchy is manually organized; automating knowledge extraction from heterogeneous sources is a promising direction. Extending KnowFeat to additional high-stakes domains (e.g., cybersecurity, insurance fraud) would further validate the generality of the approach.

References

[1] P. Domingos, "A few useful things to know about machine learning," *Communications of the ACM*, vol. 55, no. 10, pp. 78–87, 2012.

[2] T. Zhang, Z. A. Zhang, Z. Fan, H. Luo, F. Liu, Q. Liu, W. Cao, and L. Jian, "OpenFE: Automated feature generation with expert-level performance," in *Proc. of the 40th Int'l Conf. on Machine Learning (ICML)*, 2023, pp. 41 880–41 901.

[3] F. Horn, R. Pack, and M. Rieger, "The autofeat Python library for automated feature engineering and selection," in *Machine Learning and Knowledge Discovery in Databases (ECML-PKDD)*. Springer, 2019, pp. 111–120.

[4] N. Hollmann, S. Müller, and F. Hutter, "Large language models for automated data science: Introducing CAAFE for context-aware automated feature engineering," in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 36, 2023.

[5] S. Han, J. Yoon, S. O. Arik, and T. Pfister, "Large language models can automatically engineer features for few-shot tabular learning," in *Proc. of the 41st Int'l Conf. on Machine Learning (ICML)*, 2024.

[6] N. Abhyankar, P. Shojaee, and C. K. Reddy, "LLM-FE: Automated feature engineering for tabular data with LLMs as evolutionary optimizers," *Transactions on Machine Learning Research (TMLR)*, 2025.

[7] H. Li, Y. Xu, F. Zhang, R. Shi, Z. Gu, Y. Yao, Z. Liu, and Z. Li, "RiskTagger: An LLM-based agent for automatic annotation of Web3 crypto laundering behaviors," *arXiv preprint arXiv:2504.17678*, 2025.

[8] F. Nargesian, H. Samulowitz, U. Khurana, E. B. Khalil, and D. S. Turaga, "Learning feature engineering for classification," in *Proc. of the 26th Int'l Joint Conf. on Artificial Intelligence (IJCAI)*, 2017, pp. 2529–2535.

[9] J. M. Kanter and K. Veeramachaneni, "Deep feature synthesis: Towards automating data science endeavors," in *Proc. of the IEEE Int'l Conf. on Data Science and Advanced Analytics (DSAA)*, 2015, pp. 1–10.

[10] G. Katz, E. C. R. Shin, and D. Song, "ExploreKit: Automatic feature generation and selection," in *Proc. of the IEEE Int'l Conf. on Data Mining (ICDM)*, 2016, pp. 979–984.

[11] T. B. Brown, B. Mann, N. Ryder, M. Subbiah *et al.*, "Language models are few-shot learners," *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 33, pp. 1877–1901, 2020.

[12] H. Touvron, T. Lavril, G. Izacard, X. Martinet *et al.*, "LLaMA: Open and efficient foundation language models," *arXiv preprint arXiv:2302.13971*, 2023.

[13] S. Yao, J. Zhao, D. Yu, N. Du, I. Shafran, K. Narasimhan, and Y. Cao, "ReAct: Synergizing reasoning and acting in language models," in *Proc. of the Int'l Conf. on Learning Representations (ICLR)*, 2023.

[14] C. Qian, W. Liu, H. Liu, N. Chen, Y. Dang, J. Li, C. Yang, W. Chen, Y. Su, X. Cong, J. Xu, D. Li, Z. Liu, and M. Sun, "ChatDev: Communicative agents for software development," in *Proc. of the 62nd Annual Meeting of the Association for Computational Linguistics (ACL)*, 2024.

[15] S. Hong, Y. Lin, B. Liu, B. Liu, B. Wu *et al.*, "Data interpreter: An LLM agent for data science," *arXiv preprint arXiv:2402.18679*, 2024.

[16] D. A. Boiko, R. MacKnight, B. Kline, and G. Gomes, "Autonomous chemical research with large language models," *Nature*, vol. 624, pp. 570–578, 2023.

[17] Z. Jiang, D. Schmidt, D. Srikanth, D. Xu, I. Kaplan, D. Jacenko, and Y. Wu, "AIDE: AI-driven exploration in the space of code," *arXiv preprint arXiv:2502.13138*, 2025.

[18] S. Guo, C. Deng, Y. Wen, H. Chen, Y. Chang, and J. Wang, "DS-Agent: Automated data science by empowering large language models with case-based reasoning," in *Proc. of the 41st Int'l Conf. on Machine Learning (ICML)*, 2024.

[19] U. Kursuncu, M. Gaur, and A. Sheth, "Knowledge infused learning (K-IL): Towards deep incorporation of knowledge in deep learning," *Proc. of the AAAI Spring Symposium*, 2020.

[20] K. Weiss, T. M. Khoshgoftaar, and D. Wang, "A survey of transfer learning," *Journal of Big Data*, vol. 3, no. 1, pp. 1–40, 2016.

[21] Z. Chen, L. D. Van Khoa, E. N. Teoh, A. Nazir, E. K. Karuppiah, and K. S. Lam, "Machine learning techniques for anti-money laundering (AML) solutions in suspicious transaction detection: A review," *Knowledge and Information Systems*, vol. 57, no. 2, pp. 245–285, 2018.

[22] M. Weber, G. Domeniconi, J. Chen, D. K. I. Weidele, C. Bellei, T. Robinson, and C. E. Leiserson, "Anti-money laundering in Bitcoin: Experimenting with graph convolutional networks for financial forensics," *arXiv preprint arXiv:1908.02591*, 2019.

[23] Financial Action Task Force (FATF), "International standards on combating money laundering and the financing of terrorism & proliferation: The FATF recommendations," Updated 2023, 2012, available at https://www.fatf-gafi.org/en/publications/Fatfrecommendations/Fatf-recommendations.html.

[24] J. Vanschoren, J. N. van Rijn, B. Bischl, and L. Torgo, "OpenML: Networked science in machine learning," *ACM SIGKDD Explorations Newsletter*, vol. 15, no. 2, pp. 49–60, 2014.

[25] T. Chen and C. Guestrin, "XGBoost: A scalable tree boosting system," in *Proc. of the 22nd ACM SIGKDD Int'l Conf. on Knowledge Discovery and Data Mining*, 2016, pp. 785–794.

[26] S. M. Lundberg and S.-I. Lee, "A unified approach to interpreting model predictions," in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 30, 2017.

[27] G. Ke, Q. Meng, T. Finley, T. Wang, W. Chen, W. Ma, Q. Ye, and T.-Y. Liu, "LightGBM: A highly efficient gradient boosting decision tree," in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 30, 2017.